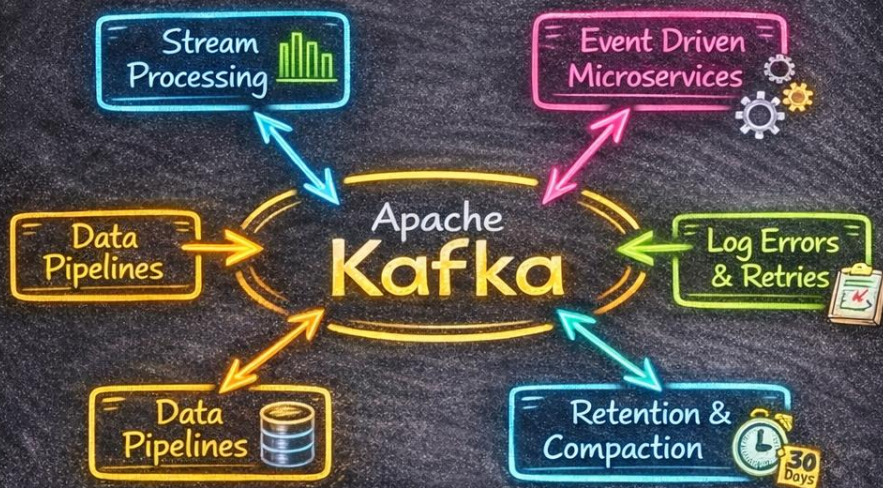


Apache Kafka

Complete Learning Guide



— Pankaj Ikhar —

Solution Architect | Mentor | Coach

— Follow for Career Growth & Interview Preparation —

Kafka

Complete Guide ★

Kafka Introduction + Real-world Use Cases

- Kafka is a distributed event streaming platform
- Built for high throughput + low latency ⚡
- Stores continuous streams of events/messages
- Supports real-time pipelines + microservices integration 📊
- Useful for analytics, monitoring, audit logs



Tip: Kafka = "log of events"
you can replay anytime ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

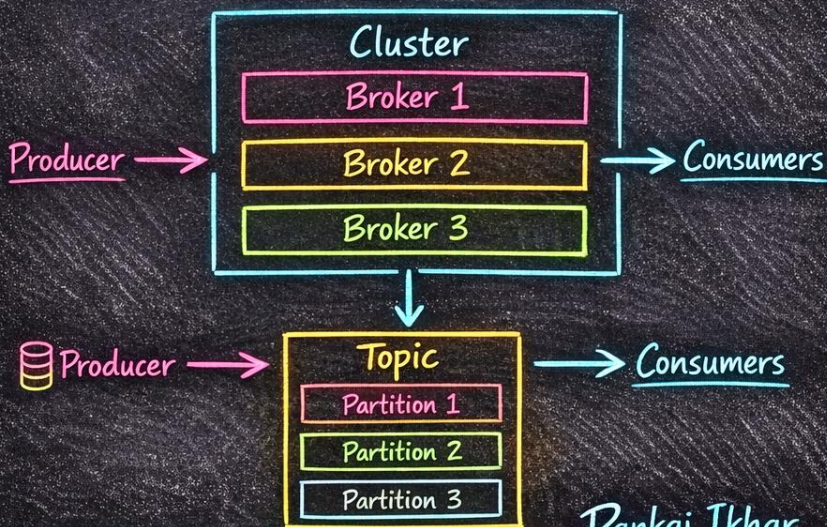
Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Kafka Architecture Overview (Cluster, Broker, Topics)

- 🌐 Cluster = group of Kafka brokers
- 🏢 Broker = Kafka server that stores partitions
- 📁 Topic = named stream of events
- 📊 Partition = scaling unit inside topic
- 👥 Replicas provide fault tolerance
- 👑 Leader handles reads/writes



Tip: More brokers + partitions
= more scale 🚀

Pankaj Ikhar

Solution Architect | Mentor | Coach

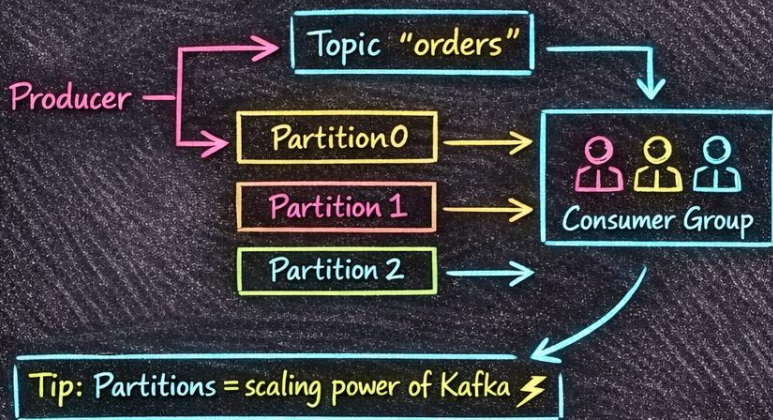
Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Events, Topics & Partitions

- 🌀 Event = record/message stored in Kafka
- 📖 Topic = stream where events are written
- 📦 Partition = sub-stream for parallelism
- 📊 More partitions = more throughput
- 🔗 Ordering guaranteed only inside partition ✓



Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Offsets & Ordering Guarantees

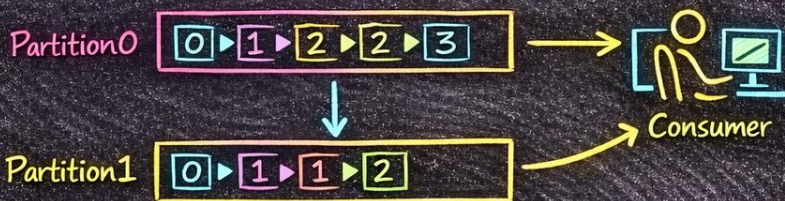
☐ Offset = message position inside partition

→ Offsets go 0,1,2,3.... sequentially

📄 Consumer tracks committed offset

✓ Ordering is per partition only ✓

📄 Different partitions have independent ordering



Consumer reads sequentially per partition

Tip: Use message key to keep same entity ordered ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Producers Basics (Send Flow)

- 🔊 Producer publishes events to Kafka topic
- 🔑 Chooses partition by key or round-robin
- 📁 Sends in batches for performance
- 🔄 Retries happen on transient failures
- 📁 Acknowledgments ensure durability



Producer is the "writer" of event log 🖋️

Tip: Producer is the "writer" of event log 🖋️

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Producer Configs (acks, retries, batch, linger, compression)

- acks=0 fastest, risk of loss
- 👉 acks=1 leader confirms
- ★ acks=all safest ✓
- ✓ retries improves reliability
- ✓ batch.size + linger.ms increase throughput
- ✓ compression reduces network load



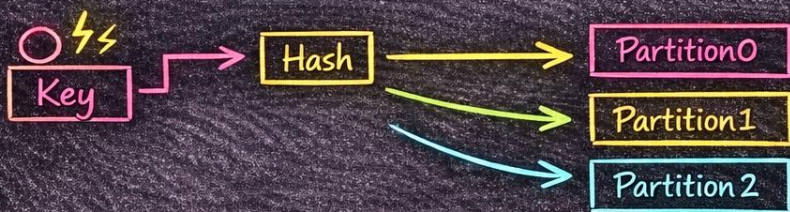
Tip: Throughput ↑ with batching + compression 🚀

Kafka

Complete Guide ★

Message Keys & Partitioning Strategy

- 🔑 Key decides partition (hashing)
- 🔑 Same key → same partition ✓
- 📁 Helps ordering for same entity
- 🎲 Null key → round-robin distribution
- 🔗 Partition strategy impacts scale & ordering



Same key → same partition

Tip: Use key = userId/orderId for correct ordering ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Consumer Basics (Poll Loop)

- 👤 Consumer reads records from partitions
- 🔄 Uses poll(i) loop to fetch messages
- 👤 Processes events then commits offset
- 📁 Pull-based model (consumer controls speed)
- 🔑 Handles retry or failure logic in app



Tip: Poll fast, process safely ✔

Pankaj Ikhar

Solution Architect | Mentor | Coach

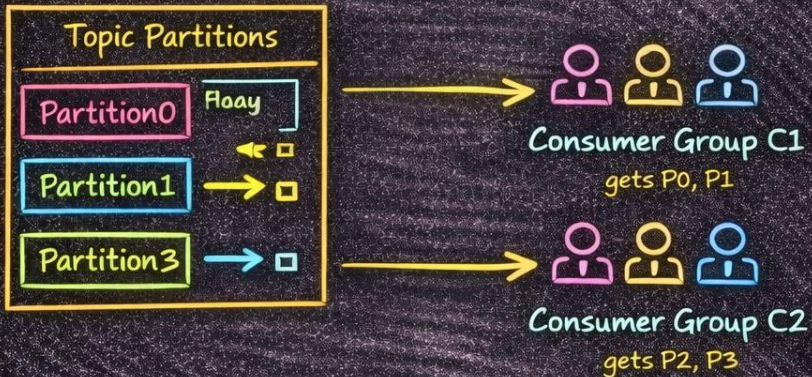
Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Consumer Groups & Partition Assignment

- 🔗 Consumer group enables scaling
- 🏠 Partitions are divided among consumers.
- 👤 One partition handled by one consumer (per group) ✓
- 📖 Multiple groups can read same topic independently
- 📡 More partitions allow more parallelism.



Tip: Poll fast, process safely ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Offset Commit (Auto vs Manual)

🔑 Auto commit = easy but risk duplicates

🔑 Manual commit = control and safety ✓

🔑 Commit after successful processing

🔑 Commit too early = message loss

🔑 Commit too late = duplicates



Tip: Commit only after DB update success ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

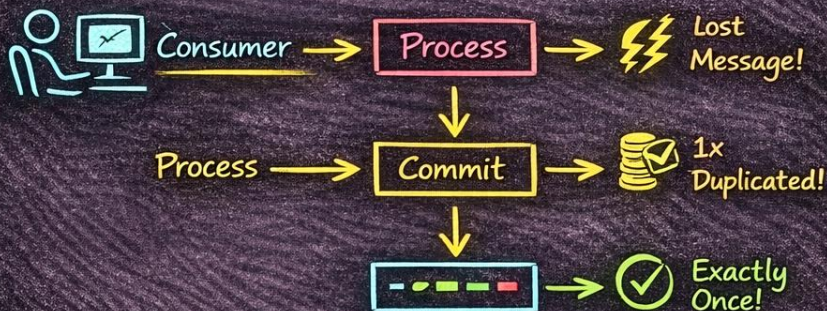
Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Delivery Semantics (At-most/At-least/Exactly-once)

- ⚡ At-most-once: no duplicates, may lose
- 🔄 At-least-once: no loss, may duplicate ✓
- 📁 Exactly-once: no loss + no duplicates ✓
- ❓ EOS needs transactions + idempotence
- 📁 Most systems use "at-least-once + dedupe"



Tip: Prefer at-least-once + idempotent consumer ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

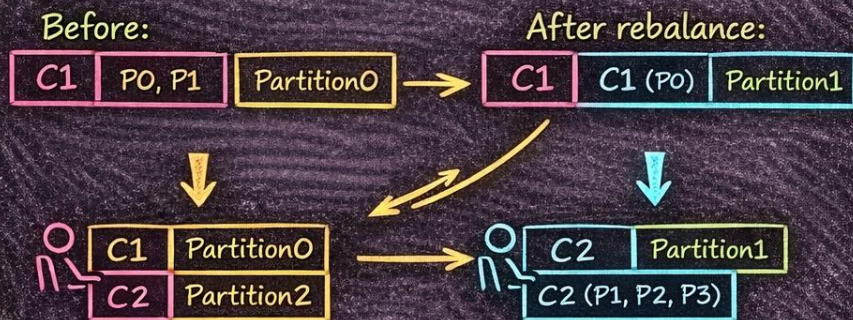
Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Rebalancing & Cooperative Rebalancing

- ✿ Rebalance = partitions reassigned in group
- 👤 Happens when consumer joins/leaves
- ⌚ Can pause processing temporarily
- ⚙️ Cooperative rebalancing reduces downtime ✓
- ⚠️ Avoid frequent rebalances in production



Tip: Stable consumers = stable throughput ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Replication Factor, Leader & Followers

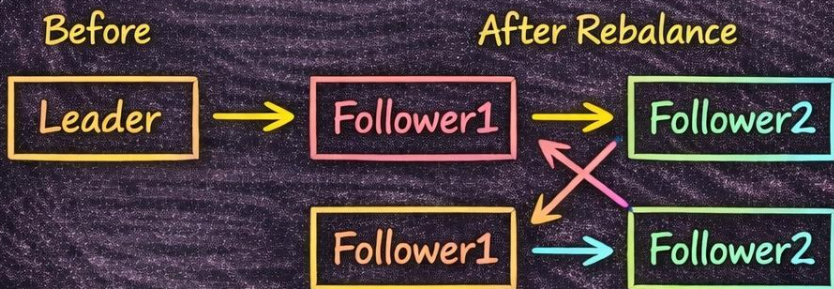
⇒ Replication factor = copies of partition

👑 Leader handles all writes/reads

👤 Followers replicate leader data

🔄 Higher RF = more fault tolerance ✓

📦 RF=3 common in production



Tip: Use RF=3 for production reliability ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

ISR, min.insync.replicas & Unclean Leader Election

»» ISR = replicas fully in sync with leader

⊕ min.insync.replicas controls write safety

⊙ With acks=all, ISR size matters ✓

✗ Unclean leader election may lose data ✗

→ Keep ISR healthy for durability



Tip: Disable unclean leader election in prod ✓

Pankaj Ikharr

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Retention Policy (Time/Size)

»» Kafka keeps messages for retention period

🕒 Time-based retention (ex: 7 days)

📦 Size-based retention (ex: 100GB)

🔄 Consumers can replay old events ✓

📁 Retention differs per topic

Topic Log

Retention Period

Leader



Old data
deleted

Tip: Kafka is not a queue-only system ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Log Compaction (Latest value per key)

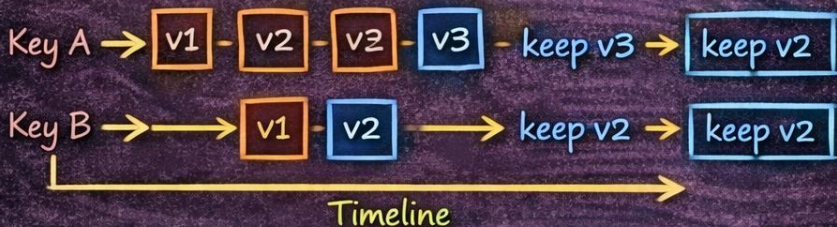
→ Compaction keeps latest message per key ✓

🕒 Removes older duplicates over time

★ Good for state topics (profile/status)

✓ Works only when key is present

✓ Not same as retention delete



Tip: Compaction = "latest state store" ✓

Kafka Complete Guide

Topic Design Best Practices

- ✓ Use meaningful topic names (`domain.event`)
- ✓ Choose partitions for scale & consumers
- ✓ Set replication factor for safety
- ✓ Set retention/compaction per use case
- ✓ Avoid too many tiny topics
- ✓ Keep message schema consistent

Topic Config

- Partitions  4
- Replication Factor  3
- Retention Policy  30 Days

Tip: Design topics like API contracts ✓

Kafka Complete Guide

Handling Duplicates + Idempotent Consumers

- ✓ Duplicates happen in at-least-once delivery
- ✓ Make consumer processing idempotent ✓
- ✓ Use unique `eventId` for dedupe
- ✓ Store processed IDs in DB/cache
- ✓ Upserts help avoid double updates

Duplicate events

- Partitions  4
- Replication Factor  3
- Retention Policy 

Tip: Consumer must be safe for retries ✓

Kafka

Complete Guide ★

Idempotent Producer & Producer Transactions

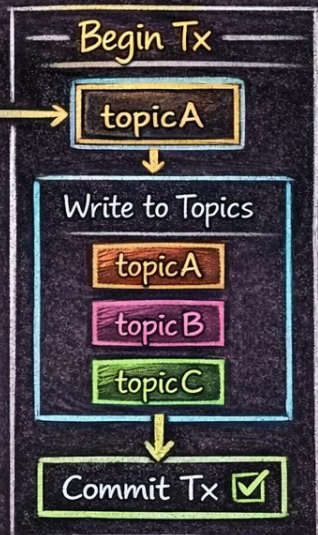
→ Idempotent producer prevents duplicates ✓

🔄 Safe retries without double writes

★ Transactions ensure atomic multi-topic writes

✓ Enables exactly-once pipelines

✓ Useful for financial / inventory systems



Tip: Use transactions for EOS streaming ✓

Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation

Kafka

Complete Guide ★

Kafka Streams Basics (KStream vs KTable)

→ Kafka Streams builds stream processing apps

☺ KStream = continuous event stream

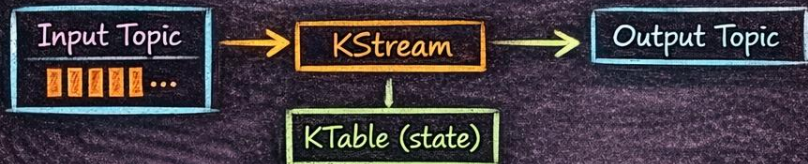
★ KTable = latest state per key ✓

✓ Streams supports joins + aggregation

✓ Works directly with Kafka topics



Tip: KTable = "database view of stream" ✓



Kafka

Complete Guide ★

Streams Operations (map/filter/join/aggregate/windowing)

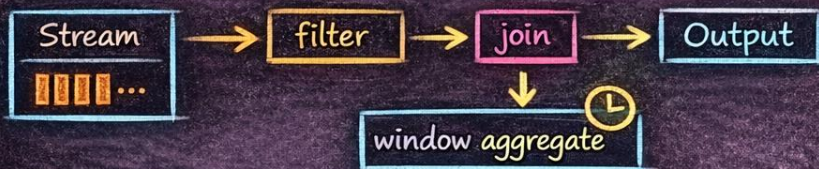
→ map/flatMap transforms events

🕒 filter removes unwanted events

★ join combines streams/tables

✓ aggregate builds summaries (count/sum)

✓ windowing groups by time (5m/1h) ✓



Tip: Windowing is key for real-time analytics ✓

Kafka

Complete Guide ★

Event Time vs Processing Time + Late Events

- Event time = when event happened
- 🕒 Processing time = when app saw it.
- ★ Late events arrive after window ends
- ✓ Use grace period to include late events ✓
- ✓ Important for correctness in analytics



Tip: Use event-time for accurate reports. ✓

Kafka

Complete Guide ★

CDC with Debezium (DB → Kafka)

→ CDC streams DB changes as events ✓

☺ Debezium reads transaction logs

★ Produces insert/update/delete events

✓ Enables real-time sync between services

✓ Useful for audit + analytics pipelines



Tip: CDC = realtime "database event feed" ✓

Kafka

Complete Guide ★

CDC with Debezium (DB → Kafka)

→ CDC streams DB changes as events ✓

☺ Debezium reads transaction logs

★ Produces insert/update/delete events

✓ Enables real-time sync between services

✓ Useful for audit + analytics pipelines



Tip: CDC = realtime "database event feed" ✓

Kafka

Complete Guide ★

Schema Registry + Avro/Protobuf/JSON

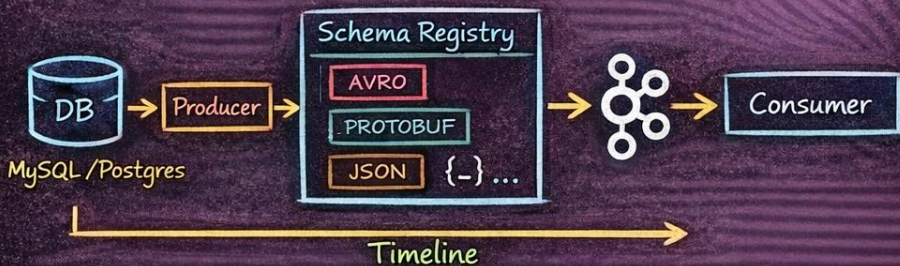
→ Schema ensures compatibility across services ✓

☺ Avro/Protobuf are compact + fast

★ JSON is easy but heavy

✓ Schema Registry manages versions

✓ Useful for audit + analytics pipelines ✓



Kafka

Complete Guide ★

Error Handling Patterns (Retry, DLT, Poison)

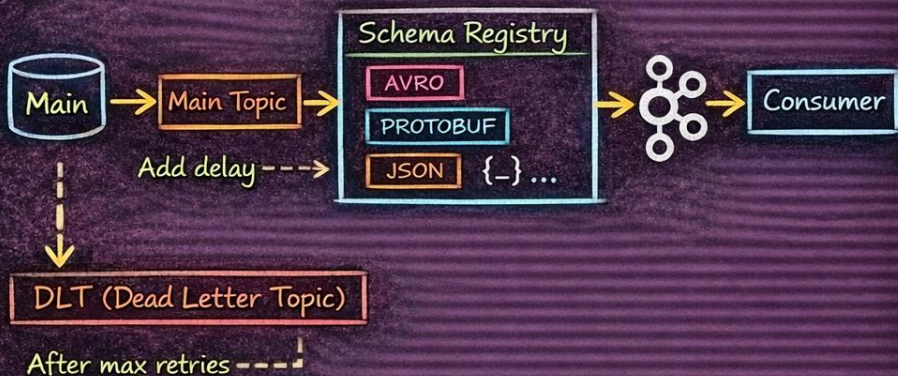
→ Retry topic handles temporary failures ✓

☑ DLT stores permanently failed messages

★ JSON is easy but heavy

✓ Schema Registry manages versions

✓ Keep failure context in headers ✓



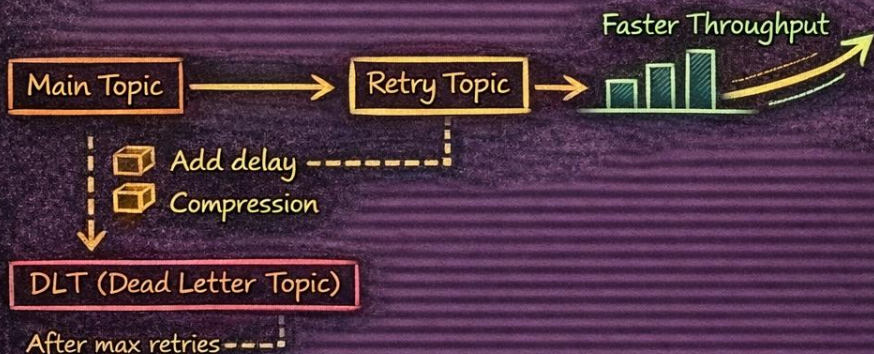
Tip: DLT prevents blocking the whole pipeline ✓

Kafka

Complete Guide ★

Performance Tuning (Throughput vs Latency)

- Increase batch.size for throughput ✓
- 🕒 linger.ms helps batching more records
- ★ compression saves bandwidth
- ✓ More partitions increases parallelism ✓
- ✓ Consumer processing speed affects lag ✓



Tip: Tune for your goal: throughput 🎯 OR ↻

Kafka

Complete Guide ★

Monitoring & Observability (Lag, Metrics, Logs)

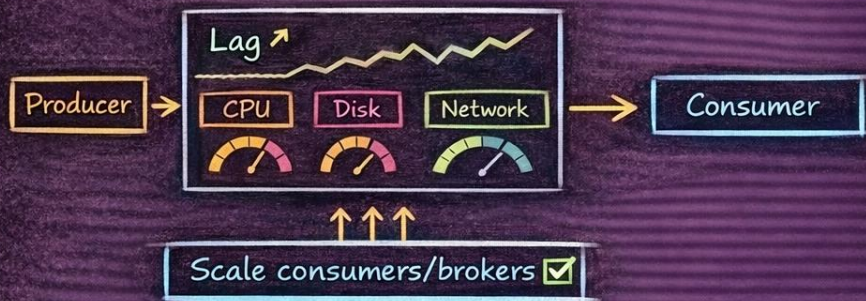
→ Consumer lag = key health metric ✓

🕒 Monitor broker CPU, disk, network

★ Track request latency & throughput

✓ Log errors, retries, rebalances

✓ Alerts prevent outages



Tip: Lag is the heartbeat of Kafka ✓

Kafka

Complete Guide ★

Security (TLS, SASL, ACLs)

- TLS encrypts data in transit ✓
- ✓ SASL authenticates clients
- ★ ACLs control who can read/write topics
- ✓ Separate prod/dev access policies
- ✓ Security is must for enterprise Kafka ✓



Tip: No security = data risk ✗ Enable TLS + ACL ✓

Kafka

Complete Guide ★

Production Deployment (KRaft, Scaling, DR)

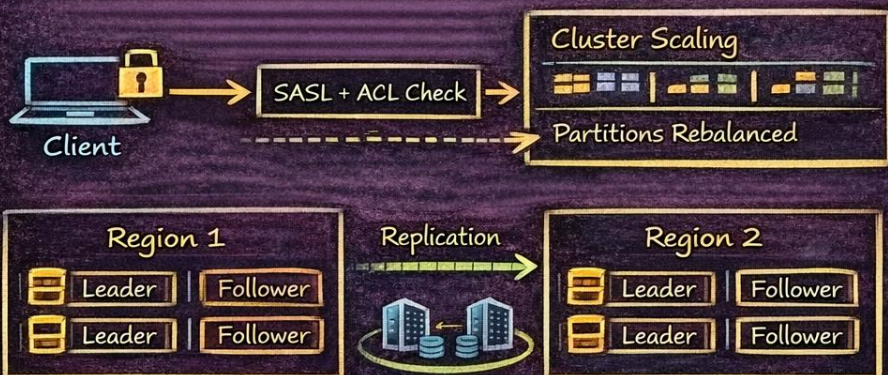
→ KRaft replaces ZooKeeper in modern Kafka ✓

🕒 Scale using brokers + partitions

★ Plan rolling upgrades safely

✓ Set replication for fault tolerance

✓ DR requires backups + multi-region strategy ✓



Tip: Production Kafka = monitoring + backups + discipline ✓

Kafka

Complete Guide ★

Quick Recap — Kafka Scaling Formula

→ More partitions = more parallelism ✓

✓ Scale using brokers + partitions

★ Lag indicates scaling need

✓ Keys help ordering + routing

✓ Replication improves durability ✓

Quick Recap — Kafka Scaling Formula



Tip: Scale Kafka logically, not blindly ✓

Kafka

Complete Guide ★

Kafka Advantages + Scaling

- High throughput = more parallelism ✓
- ⌚ One partition → one consumer per group
- ★ Lag indicates scaling need
- ✓ Keys help ordering + routing
- ✓ Replication improves durability ✓

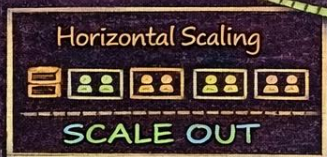
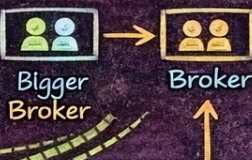
Horizontal Scaling

Most common ✓



Vertical Scaling

Add CPU + RAM



Tip: ⚖️ Horizontal = more brokers | Vertical = stronger brokers ✓

Kafka

Complete Guide ★

Kafka vs RabbitMQ /ActiveMQ

(Interview Comparison Table)

- High throughput = more parallelism ✓
- 🕒 One partition → one consumer per group
- ★ Fault tolerant via replication
- ✓ Supports multiple use cases
- ✓ Decouples producers + consumers ✓

	 Kafka	 RabbitMQ	ActiveMQ
Aspect	Kafka	RabbitMQ	ActiveMQ
Scalability	High: scales linearly	Moderate scaling	Moderate scaling
Latency	Low with batching	Low to moderate	Low to moderate
Message Ordering	Key-based ordering	Per-queue ordering	Per-queue ordering
Use Cases	Streaming, large-scale data pipelines	Queue-based persistence Transactional messages,	Queue-based persistence Transactional messages,
Resources	High CPU, disk, network	Moderate resources	Moderate resources

Tip: 🕒 Kafka = streaming | RabbitMQ = transactional 💡

Kafka

Complete Guide



Step-by-Step Learning

- ✓ Stream Processing
- ✓ Data Pipelines ✓
- ✓ Event Driven Microservices
- ✓ Log errors, retries, rebalances
- ✓ Compression reduces network load



Pankaj Ikhar

Solution Architect | Mentor | Coach

Follow for Career Growth & Interview Preparation